MyCodeSchool

for, while, funcții și liste

Fișa 1 · David · for și range, while, funcții cu return, liste ca vectori

Ce e nou față de C++: blocurile se marchează prin `:` și indentare (4 spații), fără acolade și fără `;`; `elif`; `print` cu f-string (`f"x = {x}"`); `int(input())`; funcțiile se scriu cu `def` și `return`, iar tipurile stau în semnătură: `def f(x: int) -> bool:` (`int`, `bool`, `list[int]`, `-> None` când funcția nu întoarce nimic); Python nu le verifică la rulare, dar documentează funcția. Apoi `for` cu `range(n)`, `range(a, b)`, `range(a, b, pas)` și listele, folosite ca vectorii din C++: `v = []`, `v.append(x)`, `v[i]`, `len(v)`, `for i in range(len(v)):` și `for x in v:`.
Ai voie să folosești: operatori, `if / elif / else`, `while`, funcții cu parametri și `return`, o funcție care cheamă altă funcție.
Încă NU folosim: `split`, felii (`v[a:b]`), `sum` / `max` / `min` / `sort` / `sorted` / `count` / `index` / `reverse` gata făcute, `x in v`, interschimbarea `a, b = b, a`, list comprehension, dicționare, `break`. Algoritmii îi știi din C++: aici îi scrii tu, în sintaxa Python.

Înainte de problema 1 — proiectul tău

Creezi, în același folder, fișierele `functii.py` și `main.py`. Fiecare problemă e o funcție cu numele și parametrii din enunț, care **întoarce** rezultatul cu `return`; funcția nu citește și nu afișează nimic. Citirea și afișarea stau în `main()`, în codul din „În main()”.

În `main.py` scrii `from functii import (...)`. La fiecare problemă pui numele funcției în `import` și codul ei în `main()`.

functii.py

```
def putere(a: int, n: int) -> int:
    p = 1
    ...
    return p
```

main.py

```
from functii import putere


def main() -> None:
    a = int(input())
    n = int(input())
    print(putere(a, n))


if __name__ == "__main__":
    main()
```

Citirea: `input()` citește o linie întreagă de la tastatură, fără mesaj, ca `cin`. De aceea fiecare număr se tastează pe linia lui: un vector se dă ca `n` pe prima linie, apoi câte un element pe linie. Coloana „Tastezi” arată exact liniile, în ordine.
Cum rulezi în VS Code: deschizi `main.py` și apeși ▶ (**Run Python File**) din dreapta sus, sau în terminal: `python main.py`. Tastezi valorile în terminal, fiecare urmată de `Enter`. Ce afișează trebuie să fie identic cu „Ieșire așteptată”.

Partea 1 — la clasă

Sintaxa nouă, pas cu pas. Ordinea contează: problemele de după refolosesc funcțiile de aici.

1. Puterea

for + range(n)

În `functii.py`: `def putere(a: int, n: int) -> int:`

Întoarce `a` la puterea `n`, fără `**`: înmulțești `a` de `n` ori.

1. `for i in range(n):` repetă corpul de `n` ori; `i` ia pe rând valorile `0, 1, …, n - 1`. E `for (i = 0; i < n; i++)` din C++.
2. Corpul buclei stă indentat sub `for` (4 spații), fără acolade și fără `;`.
3. Pornești cu `p = 1`, în buclă `p *= a`, după buclă `return p`.

În main()

```
a = int(input())
n = int(input())
print(putere(a, n))
```

Tastezi

```
2
10
```

Ieșire așteptată

```
1024
```

2. Suma dintr-un interval

range(a, b)

În `functii.py`: `def suma_intre(a: int, b: int) -> int:`

Întoarce suma tuturor numerelor de la `a` la `b`, inclusiv amândouă capetele.

1. `range(a, b)` merge de la `a` la `b - 1`: capătul din dreapta **nu** intră.
2. Ca să intre și `b`, scrii `for x in range(a, b + 1):`.

În main()

```
a = int(input())
b = int(input())
print(suma_intre(a, b))
```

Tastezi

```
3
8
```

Ieșire așteptată

```
33
```

3. Suma numerelor impare

range(a, b, pas)

În `functii.py`: `def suma_impare(n: int) -> int:`

Întoarce suma numerelor impare de la 1 la `n`.

1. Al treilea argument e pasul: `range(1, n + 1, 2)` dă `1, 3, 5, …`, ca `i += 2` din C++.
2. Nu ai nevoie de niciun `if`: bucla trece doar prin numerele impare.

În main()

```
n = int(input())
print(suma_impare(n))
```

Tastezi

```
9
```

Ieșire așteptată

```
25
```

4. Plecările autobuzului

range(a, b, pas)

În `functii.py`: `def numar_plecari(prima: int, limita: int, pas: int) -> int:`

Un autobuz pleacă din stație la minutul `prima`, apoi din `pas` în `pas` minute. Întoarce câte plecări are până la minutul `limita`, inclusiv.

1. `for minut in range(prima, limita + 1, pas):` trece exact prin minutele de plecare.
2. În buclă doar numeri: `plecari += 1`.

În main()

```
prima = int(input())
limita = int(input())
pas = int(input())
print(numar_plecari(prima, limita, pas))
```

Tastezi

```
5
60
15
```

Ieșire așteptată

```
4
```

5. Numărul din cifre descrescătoare

range cu pas negativ

În `functii.py`: `def numar_descrescator(a: int, b: int) -> int:`

Primești două cifre, `a > b`. Întoarce numărul format din cifrele de la `a` în jos până la `b`, inclusiv.

1. Cu pas negativ, `range` coboară: `range(a, b - 1, -1)` dă `a, a - 1, …, b`. Capătul din dreapta tot nu intră, de aceea `b - 1`.
2. Lipești fiecare cifră la coadă: `numar = numar * 10 + c`.

În main()

```
a = int(input())
b = int(input())
print(numar_descrescator(a, b))
```

Tastezi

```
9
5
```

Ieșire așteptată

```
98765
```

6. Antrenamentul de alergare

while

În `functii.py`: `def zile_pana_la_tinta(tinta: int, prima_zi: int, crestere: int) -> int:`

Un alergător aleargă `prima_zi` km în prima zi și, în fiecare zi următoare, cu `crestere` km mai mult decât în ziua dinainte. Întoarce după câte zile totalul alergat ajunge la cel puțin `tinta` km.

1. Aici nu știi dinainte câți pași sunt, deci e `while`, nu `for`: `while total < tinta:`.
2. În buclă: adaugi km-ii zilei la total, crești km-ii pentru ziua următoare, numeri ziua.

În main()

```
tinta = int(input())
prima_zi = int(input())
crestere = int(input())
print(zile_pana_la_tinta(tinta, prima_zi, crestere))
```

Tastezi

```
50
3
2
```

Ieșire așteptată

```
7
```

7. Număr prim

funcție bool

În `functii.py`: `def este_prim(x: int) -> bool:`

Întoarce `True` dacă `x` e prim și `False` altfel. În `main()`, afișezi `DA` sau `NU`.

1. Numerele mai mici ca 2 nu sunt prime: `if x < 2: return False`.
2. Cauți un divizor cu `d = 2` și `while d * d <= x:`. La primul divizor găsit, `return False` iese imediat din funcție.
3. Dacă bucla s-a terminat fără divizor, după ea: `return True`.

În main()

```
x = int(input())
if este_prim(x):
    print("DA")
else:
    print("NU")
```

Tastezi

```
97
```

Ieșire așteptată

```
DA
```

8. Câte prime într-un interval

funcție care cheamă funcție

În `functii.py`: `def cate_prime(a: int, b: int) -> int:`

Întoarce câte numere prime sunt de la `a` la `b`, inclusiv.

1. Treci prin interval cu `for x in range(a, b + 1):`.
2. Pentru fiecare `x` chemi funcția de la problema 7: `if este_prim(x):`. Nu rescrii verificarea.

În main()

```
a = int(input())
b = int(input())
print(cate_prime(a, b))
```

Tastezi

```
10
30
```

Ieșire așteptată

```
6
```

9. Citirea unui vector

listă + append

În `functii.py`: `def citeste_vector() -> list[int]:`

Citește `n`, apoi `n` numere întregi, câte unul pe linie, și întoarce lista lor. Funcția aceasta o refolosești la toate problemele de mai jos.

1. O listă goală: `v = []`. Spre deosebire de vectorul din C++, nu îi dai dimensiunea: crește singură.
2. `input()` citește o linie întreagă, ca text. Într-o buclă de `n` pași: `v.append(int(input()))` pune numărul la finalul listei.
3. `len(v)` e numărul de elemente. `print(v)` afișează lista cu paranteze drepte și virgule.

În main()

```
v = citeste_vector()
print(v)
print(len(v))
```

Tastezi

```
4
7
10
9
8
```

Ieșire așteptată

```
[7, 10, 9, 8]
4
```

10. Ziua cea mai caldă

indexare + range(len(v))

În `functii.py`: `def pozitia_maximului(v: list[int]) -> int:`

Lista `v` conține temperaturile din `n` zile. Întoarce poziția (indexul) primului maxim. În `main()` afișezi temperatura și ziua, cu zilele numerotate de la 1.

1. Indexul începe de la 0, ca în C++: primul element e `v[0]`, ultimul `v[len(v) - 1]`.
2. Pornești cu `p = 0` și parcurgi restul cu `for i in range(1, len(v)):`.
3. Doar `>` strict mută `p`, așa rămâne primul maxim.

În main()

```
v = citeste_vector()
p = pozitia_maximului(v)
print(f"Maxim {v[p]} in ziua {p + 1}")
```

Tastezi

```
5
12
17
15
17
9
```

Ieșire așteptată

```
Maxim 17 in ziua 2
```

11. Produse scumpe

for x in v

În `functii.py`: `def numar_peste(v: list[int], prag: int) -> int:`

Lista `v` conține prețurile a `n` produse. Întoarce câte produse costă strict mai mult decât `prag`. Pragul se citește după vector.

1. Când nu-ți trebuie poziția, ci doar valorile, parcurgi direct elementele: `for x in v:`. `x` ia pe rând `v[0]`, `v[1]`, …
2. Numeri cu un contor pornit de la 0.

În main()

```
v = citeste_vector()
prag = int(input())
print(numar_peste(v, prag))
```

Tastezi

```
6
45
120
80
150
99
200
100
```

Ieșire așteptată

```
3
```

12. Inversarea pe loc

modificare pe loc

În `functii.py`: `def inverseaza(v: list[int]) -> None:`

Inversează ordinea elementelor din `v` pe loc, fără altă listă. Funcția nu întoarce nimic; în `main()` afișezi lista după apel.

1. Lista primită ca parametru e **aceeași** listă din `main()`, nu o copie: ce schimbi în `v` în funcție se vede în `main()`, ca un vector transmis în C++.
2. Pui `n = len(v)` și interschimbi `v[i]` cu `v[n - 1 - i]` pentru `i` din `range(n // 2)`, cu o variabilă `aux`.
3. Funcția nu are `return` (de aceea semnătura are `-> None`); dacă ai scrie `print(inverseaza(v))`, ai vedea `None`.

În main()

```
v = citeste_vector()
inverseaza(v)
print(v)
```

Tastezi

```
5
7
9
10
6
8
```

Ieșire așteptată

```
[8, 6, 10, 9, 7]
```

Partea 2 — de exersat

Doar cerința și exemplul. Ai voie să chemi orice funcție scrisă mai sus.

13. Codul produsului

căutare

În `functii.py`: `def cauta(v: list[int], x: int) -> int:`

Lista `v` conține codurile produselor din stoc, iar după vector se citește un cod `x`. Întoarce indexul primei apariții a lui `x`, sau `-1` dacă nu apare. În `main()` afișezi `Gasit pe pozitia ...` (numerotare de la 1) sau `Nu exista`.

În main()

```
v = citeste_vector()
x = int(input())
p = cauta(v, x)
if p == -1:
    print("Nu exista")
else:
    print(f"Gasit pe pozitia {p + 1}")
```

Tastezi

```
5
104
230
517
230
999
230
```

Ieșire așteptată

```
Gasit pe pozitia 2
```

14. Cea mai rece zi

minim

În `functii.py`: `def de_cate_ori_minimul(v: list[int]) -> int:`

Lista `v` conține temperaturile minime din `n` nopți. Întoarce de câte ori apare cea mai mică temperatură, cu o singură parcurgere a listei.

În main()

```
v = citeste_vector()
print(de_cate_ori_minimul(v))
```

Tastezi

```
6
-2
5
-3
0
-3
4
```

Ieșire așteptată

```
2
```

15. Cea mai mare sumă a cifrelor

cifre

În `functii.py`: `def suma_cifrelor(x: int) -> int:` și `def cea_mai_mare_suma_cifre(v: list[int]) -> int:`

Scrie `suma_cifrelor(x)`, care întoarce suma cifrelor lui `x`, și `cea_mai_mare_suma_cifre(v)`, care întoarce elementul din `v` cu cea mai mare sumă a cifrelor. La egalitate, primul dintre ele.

În main()

```
v = citeste_vector()
print(cea_mai_mare_suma_cifre(v))
```

Tastezi

```
4
123
909
99
81
```

Ieșire așteptată

```
909
```

16. Doar numerele prime

listă nouă

În `functii.py`: `def doar_prime(v: list[int]) -> list[int]:`

Întoarce o listă nouă, cu elementele prime din `v`, în ordinea în care apar. Folosește `este_prim` de la problema 7.

În main()

```
v = citeste_vector()
print(doar_prime(v))
```

Tastezi

```
6
10
7
21
13
1
2
```

Ieșire așteptată

```
[7, 13, 2]
```

17. Palindroame

cifre

În `functii.py`: `def oglindit(x: int) -> int:` și `def cate_palindroame(v: list[int]) -> int:`

Scrie `oglindit(x)`, care întoarce numărul cu cifrele în ordine inversă, și `cate_palindroame(v)`, care întoarce câte elemente din `v` sunt egale cu oglinditul lor.

În main()

```
v = citeste_vector()
print(cate_palindroame(v))
```

Tastezi

```
5
121
45
7
1331
10
```

Ieșire așteptată

```
3
```

18. Încălzirea cea mai lungă

secvență

În `functii.py`: `def secventa_crescatoare(v: list[int]) -> int:`

Lista `v` conține temperaturile din `n` zile la rând. Întoarce lungimea celei mai lungi secvențe de zile consecutive în care temperatura a crescut strict de la o zi la alta. O zi singură e o secvență de lungime 1; dacă temperatura nu crește niciodată, răspunsul e 1.

În main()

```
v = citeste_vector()
print(secventa_crescatoare(v))
```

Tastezi

```
8
3
5
4
6
8
9
2
3
```

Ieșire așteptată

```
4
```

19. Vecini cu aceeași paritate

vecini

În `functii.py`: `def perechi_aceeasi_paritate(v: list[int]) -> int:`

Întoarce câte perechi de elemente vecine din `v` au aceeași paritate (amândouă pare sau amândouă impare).

În main()

```
v = citeste_vector()
print(perechi_aceeasi_paritate(v))
```

Tastezi

```
6
2
4
7
9
10
3
```

Ieșire așteptată

```
2
```

20. Permutare circulară

modificare pe loc

În `functii.py`: `def rotire_stanga(v: list[int]) -> None:`

Mută pe loc fiecare element cu o poziție la stânga, iar primul element ajunge pe ultima poziție. Funcția nu întoarce nimic.

În main()

```
v = citeste_vector()
rotire_stanga(v)
print(v)
```

Tastezi

```
5
1
2
3
4
5
```

Ieșire așteptată

```
[2, 3, 4, 5, 1]
```

21. Numărul din ultimele cifre

cifre

În `functii.py`: `def numar_din_ultimele_cifre(v: list[int]) -> int:`

Întoarce numărul format din ultimele cifre ale elementelor lui `v`, în ordinea elementelor.

În main()

```
v = citeste_vector()
print(numar_din_ultimele_cifre(v))
```

Tastezi

```
4
123
45
7
60
```

Ieșire așteptată

```
3570
```

22. Pozițiile impare

range cu pas pe indici

În `functii.py`: `def suma_pozitii_impare(v: list[int]) -> int:`

Lista `v` conține cheltuielile dintr-o lună. Întoarce suma elementelor de pe pozițiile 1, 3, 5, … (numerotate de la 1), cu o buclă care trece doar prin acele poziții.

În main()

```
v = citeste_vector()
print(suma_pozitii_impare(v))
```

Tastezi

```
6
10
3
20
5
30
1
```

Ieșire așteptată

```
60
```

23. Șir strict crescător

bool

În `functii.py`: `def este_strict_crescator(v: list[int]) -> bool:`

Întoarce `True` dacă fiecare element din `v` e strict mai mare decât cel dinaintea lui, altfel `False`. În `main()` afișezi `DA` sau `NU`.

În main()

```
v = citeste_vector()
if este_strict_crescator(v):
    print("DA")
else:
    print("NU")
```

Tastezi

```
4
2
5
5
8
```

Ieșire așteptată

```
NU
```

24. Egal cu suma vecinilor

vecini

În `functii.py`: `def egale_cu_suma_vecinilor(v: list[int]) -> int:`

Întoarce câte elemente din `v`, în afară de primul și de ultimul, sunt egale cu suma celor doi vecini ai lor.

În main()

```
v = citeste_vector()
print(egale_cu_suma_vecinilor(v))
```

Tastezi

```
6
1
3
2
5
3
8
```

Ieșire așteptată

```
2
```

Partea 3 — provocări

Doar cerința.

25. Cifra cea mai frecventă

provocare

În `functii.py`: `def cifra_cea_mai_frecventa(v: list[int]) -> int:`

Lista v conține numere naturale. Întoarce cifra care apare de cele mai multe ori în scrierea tuturor numerelor din v. La egalitate, cifra cea mai mică.

În main()

```
v = citeste_vector()
print(cifra_cea_mai_frecventa(v))
```

Tastezi

```
3
1223
452
29
```

Ieșire așteptată

```
2
```

26. Cel mai bun interval de profit

provocare

În `functii.py`: `def suma_maxima_secventa(v: list[int]) -> int:`

Lista v conține profitul (pozitiv sau negativ) al unui magazin în n zile. Întoarce cea mai mare sumă a unei secvențe de zile consecutive, cu o singură parcurgere a listei.

În main()

```
v = citeste_vector()
print(suma_maxima_secventa(v))
```

Tastezi

```
8
3
-5
4
-1
2
6
-8
1
```

Ieșire așteptată

```
11
```

27. Interclasare

provocare

În `functii.py`: `def interclasare(a: list[int], b: list[int]) -> list[int]:`

Primești două liste ordonate crescător. Întoarce o listă nouă, ordonată crescător, cu toate elementele din amândouă, cu o singură trecere prin fiecare listă. Se citește întâi lista a, apoi lista b.

În main()

```
a = citeste_vector()
b = citeste_vector()
print(interclasare(a, b))
```

Tastezi

```
3
1
4
9
4
2
3
10
12
```

Ieșire așteptată

```
[1, 2, 3, 4, 9, 10, 12]
```

28. Fără dubluri

provocare

În `functii.py`: `def fara_duplicate(v: list[int]) -> list[int]:`

Întoarce o listă nouă cu elementele din v, fiecare o singură dată, în ordinea primei apariții.

În main()

```
v = citeste_vector()
print(fara_duplicate(v))
```

Tastezi

```
6
5
3
5
1
3
5
```

Ieșire așteptată

```
[5, 3, 1]
```

29. Zerourile la final

provocare

În `functii.py`: `def zerourile_la_final(v: list[int]) -> None:`

Mută pe loc toate elementele egale cu 0 la finalul lui v, păstrând ordinea celorlalte, fără altă listă. Funcția nu întoarce nimic.

În main()

```
v = citeste_vector()
zerourile_la_final(v)
print(v)
```

Tastezi

```
6
0
4
0
0
7
2
0
```

Ieșire așteptată

```
[4, 7, 2, 0, 0, 0]
```

30. Al doilea cel mai mare punctaj

provocare

În `functii.py`: `def al_doilea_maxim(v: list[int]) -> int:`

Lista v conține punctajele unor elevi, numere naturale. Întoarce a doua cea mai mare valoare **distinctă** din v, sau −1 dacă nu există, cu o singură parcurgere a listei.

În main()

```
v = citeste_vector()
print(al_doilea_maxim(v))
```

Tastezi

```
5
7
10
9
10
3
```

Ieșire așteptată

```
9
```